

How the Heart Attack Detection Model Works

A plain-English guide to the data, the training, and the scores
IoT-Based Heart Attack Pattern Detection Using Deep Learning

What the model does

The model looks at **10 seconds of an ECG signal** from a single sensor and answers one question:

Is this a normal heart, or is there a heart attack (myocardial infarction, MI) pattern?

It is a **two-class** problem. The model does not give a full diagnosis. It gives one number between 0 and 1, which is how sure it is that the signal shows an MI pattern. 0.02 means "almost certainly normal". 0.95 means "very likely MI".

Step 1 | Separating the data

Where the data comes from

We use **PTB-XL**, a free hospital dataset from PhysioNet. It has **21,799 ECG recordings**. Each recording is 10 seconds long and was labelled by real doctors.

Which recordings we keep

We do not use all of them. We keep only the recordings that are **clearly one thing or the other**, so the two groups stay clean:

Group	What it means	How many
Normal (label 0)	The only finding is 'normal ECG'	9,069
Heart attack (label 1)	The record has an MI finding	5,469
Not used	Other heart diseases, or unclear cases	7,261

Total used for the project: **14,538 recordings**.

Important to say in the thesis: throwing away those 7,261 records makes the job easier than real life. The model only ever learns to tell MI apart from a *healthy* heart, never from a *different* disease. A real device would meet those patients too.

Which part of the signal we use

A hospital ECG records 12 different views, called leads. Our cheap device (ESP32 + AD8232) has only two electrodes, so it can realistically record **Lead I** only. To make the test fair, we throw away the other 11 leads and train only on Lead I.

Each recording becomes **1,000 numbers** (100 samples per second x 10 seconds).

Splitting into three groups

This is the most important step for an honest result. We split the recordings into three separate groups:

Group	What it is for	Records
Training (folds 1-8)	The model learns from these	11,622
Validation (fold 9)	Decides when to stop, and picks the threshold	1,454

Test (fold 10)	The final exam. Used ONCE, at the very end	1,462
----------------	--	-------

PTB-XL already gives every recording a fold number from 1 to 10. We use those official numbers instead of splitting randomly, because they are built so that **the same patient never appears in two groups**. We checked this ourselves: **0 patients** appear in more than one fold.

Why this matters: if the same patient were in both training and test, the model could simply remember that person. The score would look great but would be a lie. This mistake is called **data leakage**.

Step 2 | Training the model

Cleaning the signal

Every recording is **z-score normalised**: we subtract its average and divide by its spread, so every signal ends up with average 0 and spread 1.

```
cleaned = (signal - average) / (spread + 0.00000001)
```

Different machines record at different strengths. This step makes the model look at the **shape** of the heartbeat instead of how big the numbers are.

The network

We use a **1D Convolutional Neural Network (1D CNN)**. A normal CNN slides small filters over a picture. A 1D CNN slides them along a signal, so it can find a shape (like a heartbeat spike) wherever it happens in the 10 seconds.

The network has 4 blocks. Each block does the same four things:

- **Convolution** - slide filters along the signal to find patterns
- **Batch normalisation** - keep the numbers in a healthy range so training is stable
- **ReLU** - lets the network learn curved, not just straight-line, patterns
- **Max pooling** - halve the length, keeping the strongest responses

Early blocks see small pieces of time. Later blocks see whole heartbeats. At the end, the network squeezes everything into one number between 0 and 1. In total it has **229,473 learnable settings** (parameters).

Handling the uneven groups

There are more Normal records (9,069) than MI records (5,469). Without help, the model could cheat by always answering "Normal". We use **class weights**, which make each mistake on an MI record cost more, so that trick stops working.

Knowing when to stop

After every pass through the data, we check the model on the **validation** group. When it stops getting better there for 12 passes in a row, training stops and we go back to the best version. This is called **early stopping**, and it stops the model from memorising the training data.

Two extra improvements

- **Augmentation** - during training we slightly change each signal every time: shift it in time, add slow drift (like breathing), and add small electrical noise. This copies the kind of mess a cheap electrode makes.
- **Ensemble of 5** - we train the same network 5 times from different random starts and average their 5 answers. Mistakes are partly random, so averaging cancels some of them out.

Step 3 | Testing the model

Now we use the **test group (fold 10)**, the 1,462 recordings the model has never seen. It contains **550 heart attacks** and **912 normal** records.

The confusion matrix

Every answer falls into one of four boxes. These four numbers are where every score comes from.

	Model says NORMAL	Model says MI
Truly NORMAL	TN = 609 Correct all-clear	FP = 303 False alarm
Truly MI	FN = 47 MISSED HEART ATTACK	TP = 503 Correctly caught

These numbers are the ensemble at its high-recall setting.

FN (bottom-left) is the dangerous box. Those are real heart attacks the model called normal. In a health system this is the worst mistake, so the whole setup is arranged to keep this number small.

Step 4 | The scores explained

Accuracy

$$\text{Accuracy} = (\text{TP} + \text{TN}) / \text{everything} = (503 + 609) / 1462 = 0.761$$

In words: out of all the answers, how many were right.

Warning: accuracy is misleading here. Our test group is 62.4% normal. So a lazy model that answers "Normal" to *everything* already scores **62.4%** accuracy - while catching zero heart attacks. Never judge this project by accuracy alone.

Precision

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) = 503 / (503 + 303) = 0.624$$

In words: when the model shouts "heart attack!", how often is it right? Here, about 6 out of every 10 alarms are real.

Recall (also called Sensitivity)

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) = 503 / (503 + 47) = 0.915$$

In words: out of all the *real* heart attacks, how many did we catch? We caught 503 out of 550.

This is the most important score in this project. Missing a real heart attack is far more dangerous than sending someone for an unnecessary check. That is why we tune the system to make recall high, even though it costs accuracy.

Specificity

$$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP}) = 609 / (609 + 303) = 0.668$$

In words: out of all the *healthy* people, how many did we correctly leave alone. Recall and specificity must always be read together: a model that says "MI" to everyone has perfect recall and useless specificity.

F1-score

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) = 0.742$$

In words: one fair number that balances precision and recall. It only goes high if *both* are decent, so it cannot be fooled the way accuracy can.

ROC-AUC

$$\text{ROC-AUC} = 0.908 \quad (1.0 = \text{perfect}, 0.5 = \text{random guessing})$$

In words: if you pick one random MI record and one random normal record, this is the chance the model gives the MI record the higher score. It does not depend on any cut-off, so it measures the **true quality of the model**. This is the number we try to improve.

PR-AUC

$$\text{PR-AUC} = 0.894$$

In words: the same idea as ROC-AUC, but it ignores the easy correct all-clears. That makes it the fairer headline number when the two groups are uneven, as they are here.

The threshold: turning a number into a decision

The model outputs a probability. To give a yes/no answer we need a cut-off. Most people use 0.5 without thinking, but **0.5 is just a habit, not a rule** - and for medicine it is the wrong choice.

We choose the cut-off using the **validation** group, then apply it unchanged to the test group. Choosing it on the test group would be cheating.

Cut-off	Recall	Precision	Accuracy	Missed MI	False alarms
0.50 (default)	0.735	0.852	0.852	146	70
0.53 (best F1)	0.704	0.868	0.848	163	59
0.30 (high recall)	0.915	0.624	0.761	47	303

Read the last two columns carefully. Moving the cut-off from 0.50 down to 0.30 means we **miss 47 heart attacks instead of 146** - almost 100 more lives reached. The price is more false alarms (303 instead of 70), and the accuracy actually goes *down* from 0.852 to 0.761.

This is the key lesson of the project: the setting that is best for patients has the *worst* accuracy. If we had tuned for accuracy, we would have built a more dangerous device.

Final results

Model	ROC-AUC	Recall	Missed MI
Single CNN (baseline)	0.9017	0.896	57
Ensemble of 5 (final)	0.9076	0.915	47

Both measured on the test fold at their own high-recall setting.

The improvement is small but **real**: we tested it by resampling the test set 2,000 times, and the difference was +0.0059 with a 95% confidence range of +0.0009 to +0.0110. Because that range does not include zero, we can honestly say the ensemble is better.

Honest limits

- **One lead cannot see everything.** Lead I looks at the heart from one side. Heart attacks in the lower (inferior) or back (posterior) wall are almost invisible to it. This is the main reason the score is not higher.
- **Most MI records are old, not fresh.** Only about 6% of our MI records carry a sign of an *acute* attack. So the model mostly recognises the scar of a *past* heart attack. It is a screening tool, not an emergency alarm.
- **Clean hospital data.** The model was trained on tidy hospital recordings. Signals from the real ESP32 device will be noisier.
- **Alarms would be rarer in real life.** Our test group is 37.6% heart attacks. In the general public it would be far lower, so most alarms would be false. The device should tell someone to *get checked*, never give a diagnosis.
- **Not a medical device.** This is an academic research prototype.